Bioinformatics, Original Paper, Sequence analysis — draft

dinara-align: hardware-aware algorithms for exact pairwise alignment

Konstantinos Kyriakidis[1,*] and Benedict Paten[1,*]
[1]UC Santa Cruz Genomics Institute, Santa Cruz, CA, USA
[*]To whom correspondence should be addressed: kkyriaki@ucsc.edu, bpaten@ucsc.edu

Motivation: Pairwise alignment underlies read mapping, genome assembly, variant calling and pangenome construction. Heuristic aligners can miss or misplace alignments with large indels or clustered errors, whereas exact aligners return an optimal alignment whatever the data, which makes their results reproducible and a reference for evaluating heuristics. Yet no single exact aligner is fastest across workloads: diagonal transition wins on short and near-identical pairs, seeded band doubling on long nanopore reads, and SIMD methods on batches of short pairs.
Results: We present dinara-align, an exact aligner that chooses its algorithm per pair, making every choice by a rule that either provably preserves optimality or bounds the work it can waste. A gap-affine alignment starts with one search and adds a second only when its measured work exceeds what two would need, which we prove loses no optimal alignment; searches abandon a projection only after spending a fixed share of the alternative's cost; short pairs are batched in narrow bands certified per vector lane; and inexact seed neighbours are inserted without search, provably leaving pruning unchanged. Every engine returns the same alignment among equally optimal ones. On an Intel Skylake-X processor, dinara-align was the fastest of the exact aligners compared on every dataset, from 250 bp reads to 1 Mbp pairs and from 0 to 20% divergence: up to 2.4 times faster than A*PA2 on ultra-long nanopore reads, 2.0 and 5.2 times faster than WFA on Illumina and HiFi reads, and 1.1 to 3.8 times faster than WFA and BiWFA at gap-affine costs.
Availability and implementation: dinara-align is free software under the Mozilla Public License 2.0, available at `https://github.com/kokyriakidis/dinara-align` with C, C++, Python and command-line interfaces.
Contact: kkyriaki@ucsc.edu, bpaten@ucsc.edu
Supplementary information: Supplementary data are available at *Bioinformatics* online.

1 Introduction

Pairwise alignment is a building block of read mapping, genome assembly and variant calling. The classical dynamic program computes an optimal alignment over an $n \times m$ matrix (Gotoh, 1982), practical for short reads but not for long ones: nanopore reads span tens to hundreds of kilobases with 5 to 15% errors, and one pair of 500 kbp reads fills 2.5×10^{11} cells.

Heuristic aligners compute only part of the matrix, a band, a window or the cells before a drop in score, and lose the guarantee of optimality: their result depends on these cut-offs, and alignments with large indels or clustered errors can be missed or misplaced. Exact aligners instead skip only those cells that provably cannot lie on an optimal path, so that their running time depends on the distance between the sequences rather than on the product of their lengths.

Three ideas underlie current exact aligners. *Band doubling* computes only the diagonals a path within a cost bound can reach, doubling the bound until the optimum lies within it (Ukkonen, 1985); *bit-parallelism* packs 64 cells of a column into a machine word (Myers, 1999), and Edlib combines the two (Šošić and Šikić, 2017). *Diagonal transition* extends the furthest-reaching paths one cost at a time, following matches at no cost (Ukkonen, 1985; Myers, 1986); WFA generalised it to gap-affine costs (Marco-Sola et al., 2021) and BiWFA made its memory linear in the distance (Marco-Sola et al., 2023). A*PA added A* search with a gap-chaining seed heuristic (Groot Koerkamp and Ivanov, 2024), and A*PA2 combined this heuristic with band doubling over SIMD blocks, aligning ultra-long reads up to 19 times faster than Edlib and BiWFA (Groot Koerkamp, 2024); QuickEd bounds the distance heuristically and then aligns within the bound (Doblas et al., 2025). A second line of work vectorises the full dynamic program, with anti-diagonal (Wozniak, 1997), striped (Farrar, 2007; Zhao et al., 2013) and prefix-scan (Daily, 2016) layouts, difference recurrences (Suzuki and Kasahara, 2018; Li, 2018), adaptive blocks (Liu and Steinegger, 2023) and inter-sequence parallelism (Rognes, 2011; Rahn et al., 2018), at a cost that grows with the matrix or the band rather than with the distance.

Each of these methods is fastest on part of the range of workloads: diagonal transition, whose work grows with the square of the distance, on short and near-identical pairs; seeded band doubling, whose work grows with the length, on long divergent reads; and inter-sequence vectorisation on batches of short pairs. An aligner that is fastest across workloads must therefore choose among them for each pair, and an exact aligner must make every such choice, which algorithm, which bound and when to abandon a search, without giving up optimality. dinara-align makes each choice by a rule that either provably preserves optimality or bounds the work the choice can waste, and implements each algorithm for the latency and memory behaviour of current processors. Our contributions are:

1. **One wavefront search or two, chosen by measured work** (Section 3.9). A gap-affine alignment starts with one search and adds a second only when its measured work exceeds what two would need, without losing any optimal alignment (Proposition 2).

2. **Projections used with guarantees** (Sections 3.3 and 3.10). A projected distance sets a band's

first bound only when two independent estimates agree, and a search abandons a projection only after spending a fixed share of the alternative's cost.

3. **Certified bands for inter-sequence batches** (Section 3.6). Each short pair is aligned in a vector lane within a narrow band that the lane certifies optimal (Proposition 1), adapting the speculate-and-verify approach of SeedEx (Fujiki et al., 2020) to software.
4. **Direct neighbour construction for the seed heuristic** (Section 3.5). Inexact neighbours of exact seed matches are inserted without search, leaving local pruning provably unchanged (Lemma 1).
5. **One alignment whichever engine computes it** (Sections 3.1 and 3.7). All engines follow one rule among equally optimal alignments, and a traceback past the memory limit recomputes stretches from recorded state rather than splitting the pair.
6. **Engines arranged for the processor** (Sections 3.2, 3.4 and 3.11): gathered diagonal transition, an interleaved bit-parallel sweep with a shorter dependency chain, and no per-pair constant costs on short reads.

We evaluate dinara-align on the A*PA2 and QuickEd benchmarks, to which we add real Illumina and PacBio HiFi reads paired with the genome they came from. Sections 2 to 5 give the background, the methods, the results and the discussion.

2 Background

2.1 Alignment and costs

Let $A = a_1 \dots a_n$ be the reference and $B = b_1 \dots b_m$ the query, and let cell (i, j) lie on diagonal $j - i$. Under gap-affine costs a mismatch costs x and a gap of k letters $g(k) = o + ke$; unit costs have $x = e = 1$ and $o = 0$, two-piece costs take the minimum of two such functions (Li, 2018), and a substitution matrix instead scores each pair of letters. An alignment is *exact* if it is optimal under the requested costs, mode (global, ends-free, extension or local), band and cost limit. At unit costs the distance $D(i, j)$ between the prefixes $a_1 \dots a_i$ and $b_1 \dots b_j$ satisfies

$$D(i,j) = \min\big(D(i-1,j)+1,\ D(i,j-1)+1,\ D(i-1,j-1)+[a_i \neq b_j]\big), \tag{1}$$

with $D(i, 0) = i$ and $D(0, j) = j$, and the distance is $D(n, m)$; gap-affine costs need three such layers (Gotoh, 1982).

2.2 Band doubling and bit-parallel computation

An alignment of cost at most t visits only diagonals d with $|d| + |d - \delta| \leq t$, where $\delta = m - n$; band doubling computes only those cells, doubling t until $D(n, m) \leq t$ (Ukkonen, 1985). Myers' recurrence advances 64 cells of a column per machine word (Myers, 1999), and A*PA2 sweeps the band in tiles of 256 columns of vector registers, dropping rows that a score already rules out (Groot Koerkamp, 2024).

2.3 Diagonal transition

Diagonal transition keeps, for each cost s and diagonal, the furthest cell an alignment of cost s reaches, extended along matches at no cost, in $O(n + s^2)$ steps (Ukkonen, 1985; Myers, 1986); WFA generalises it to gap-affine costs with three wavefronts per cost (Marco-Sola et al., 2021), and BiWFA runs it from both ends (Marco-Sola et al., 2023).

2.4 The seed heuristic

A*PA divides the reference into disjoint seeds of k bases and finds their matches in the query, exactly or with one edit (Groot Koerkamp and Ivanov, 2024). Its gap-chaining seed heuristic bounds the cost from a cell to the end of the alignment by the seeds ahead that no chain of matches from that cell covers, counting the gaps between chained matches, and never overestimates. Local pruning discards a match ν unless $c(\nu)+h(e(\nu)) < \tau$, where $c(\nu)$ is its cost, $e(\nu)$ its end cell, $h(u)$ the least cost of a path from u across the following seeds, and τ a threshold that depends only on the seeds. A*PA2 uses the heuristic to drop the rows of each band whose score plus the heuristic exceeds the bound (Groot Koerkamp, 2024).

3 Methods

3.1 Overview

Each problem is assigned to an algorithm (Table 1; Fig. 1). Unit costs follow A*PA2 (Groot Koerkamp, 2024): band doubling over 256-column tiles of bit-parallel words, pruned by the gap-chaining seed heuristic (Groot Koerkamp and Ivanov, 2024), with near-identical pairs aligned by diagonal transition alone. Gap-affine costs use the wavefront of WFA and BiWFA (Marco-Sola et al., 2021, 2023). A short diagonal-transition search estimates the distance, which with the length and the cost model selects the algorithm.

Under costs every engine selects the same alignment among equally good ones, following the backtrace order of WFA2-lib: from the end, an edit is taken into a cell whenever one is optimal, a substitution before a deletion before an insertion, and a gap is extended before it is opened. The tile traceback applies this order cell by cell (Section 3.7), the batches and the difference sweep store per-cell flags under it, and the wavefronts take the furthest-reaching source, which makes the same choice; leftmost indels come from the same rule applied to both sequences reversed. Two exceptions remain: a gap-affine pair whose wavefronts would exceed the memory limit is split and follows the rule within each

piece, and alignments under a substitution matrix follow the traceback order of Gotoh's recurrence.

3.2 A shorter critical path for Myers' recurrence

As in Edlib and A*PA2 (Šošić and Šikić, 2017; Groot Koerkamp, 2024), the query is packed 64 bases per word, and Myers' recurrence (Myers, 1999) advances a word by one column. With match mask E, vertical difference masks P_v, M_v and the incoming negative horizontal difference folded into $E' = E \vee M_h^{\text{in}}$, A*PA2 computes

$$\begin{aligned} c &= (E' \wedge P_v) + P_v, \\ X_h &= (c \oplus P_v) \vee E', \\ P_h &= M_v \vee \neg(X_h \vee P_v), \\ M_h &= P_v \wedge X_h. \end{aligned} \tag{2}$$

The horizontal masks P_h and M_h determine the vertical masks of the next column, so the path from c to them lies on the loop-carried chain. Since $(c \oplus P_v) \vee P_v = c \vee P_v$ and $P_v \wedge (c \oplus P_v) = P_v \wedge \neg c$, the same masks are obtained as

$$\begin{aligned} P_h &= M_v \vee \neg\big(c \vee (E' \vee P_v)\big), \\ M_h &= (P_v \wedge \neg c) \vee (P_v \wedge E'), \end{aligned} \tag{3}$$

where $E' \vee P_v$ and $P_v \wedge E'$ do not depend on c and are computed off the critical path. This removes two operations from the loop-carried chain and, for a single word, reduces the cost from 14 to 8 cycles per column on the Skylake-X. Myers (Myers, 1999), Edlib and A*PA2 use the original form, and we are not aware of a previous description of the regrouped one.

Words occupy the lanes of a vector, staggered by one column, so that the horizontal difference leaving one word enters the next through a lane rotation (Wozniak, 1997; Groot Koerkamp, 2024), and each column waits for the rotation of the previous one. A*PA2 overlaps these waits with two four-lane vectors; on AVX-512, where a vector holds eight words, dinara-align interleaves two eight-word groups, the lower ten columns behind the upper, so that each group fills the latency of the other, and bands of two or three words run in general-purpose registers, whose operations have lower latency. The sweep takes about 2.4 cycles per word and column.

3.3 Band bounds from agreeing estimates

Band doubling is exact for any initial bound, but its running time depends on how close that bound is to the distance. A computation that has reached cost s at anti-diagonal r extrapolates the distance $\hat{d} = s(n+m)/r$, which is reliable when edits are spread evenly and too high by orders of magnitude when they gather, as at the ends of nanopore reads. dinara-align therefore sets the first bound from two extrapolations, $\hat{d}_1$ from the first eight edits of a diagonal-transition search and $\hat{d}_2$ from the same search at its budget, only if $\max(\hat{d}_1, \hat{d}_2) \le \frac{4}{3}\min(\hat{d}_1, \hat{d}_2)$; otherwise, as in A*PA2, the bound starts 256 above the known lower bound and each retry at most doubles it. A failed round aims the next bound from where it stopped, its progress extrapolated to the whole width, and within a round the distance is extrapolated again after one eighth, one quarter and one half of the columns, lowering the bound or abandoning the round. These rules change only the sequence of bounds, so the result stays exact.

Within a round, the band reads the scores down the right edge of each tile and drops the rows at its top and bottom whose score plus the heuristic exceeds the bound, as A*PA2 does. A score changes by at most one per row, and the heuristic by at most its climb c, one for exact seeds and two for inexact ones, so a row that exceeds the bound by x rules out the next $\lceil x/(1+c) \rceil$ rows without reading them. The bottom of the next tile is bounded by the deepest row kept, since going k rows past it costs at least k more. With seeds, when at least one seed in 32 chains from the origin, the first bound starts 64 above the heuristic at the origin, which on long reads lies within a few percent of the distance.

3.4 Diagonal transition for near-identical pairs

Diagonal transition extends each diagonal of a wavefront along its run of matches. dinara-align compares eight letters at a time by the exclusive-or of two 64-bit words and a count of trailing zeros. On AVX-512, eight diagonals are extended together: their letters are loaded with gather instructions, and the wavefront is padded with unreachable diagonals so that every step processes full vectors under branch-free masks. The gather masks are derived from the lanes of each group, which avoids a dependency between consecutive gathers that would otherwise serialise them. The same vectorised step serves near-identical pairs, computed from one or both ends (Marco-Sola et al., 2023), the traceback inside each tile, restricted to the diagonals that can still reach the traced cell, and the three-layer gap-affine wavefront.

A pair whose diagonal transition finishes within its budget is aligned from its wavefronts alone. From one end the wavefronts are kept whole. From both ends, as in BiWFA, the forward wavefronts are grown on to the proven distance, each cost kept only on the diagonals on which the backward wavefronts show that an optimal path passes, so that the extra costs grow few diagonals. The backtrace then takes, at each cost, the furthest-reaching source, which is the order of Section 3.1, and the CIGAR is written run by run from the wavefronts.

3.5 Seed heuristic: direct neighbour construction

The reference is divided into disjoint seeds of 12 bases, matched exactly, or of 16 bases, matched with at most one edit (Groot Koerkamp and Ivanov, 2024). A 16-base seed that matches with at most one edit matches one of its two 8-base halves exactly, so each half indexes a table. Since these tables exceed the caches, the lookups for 16 query rows are issued together, so that their memory

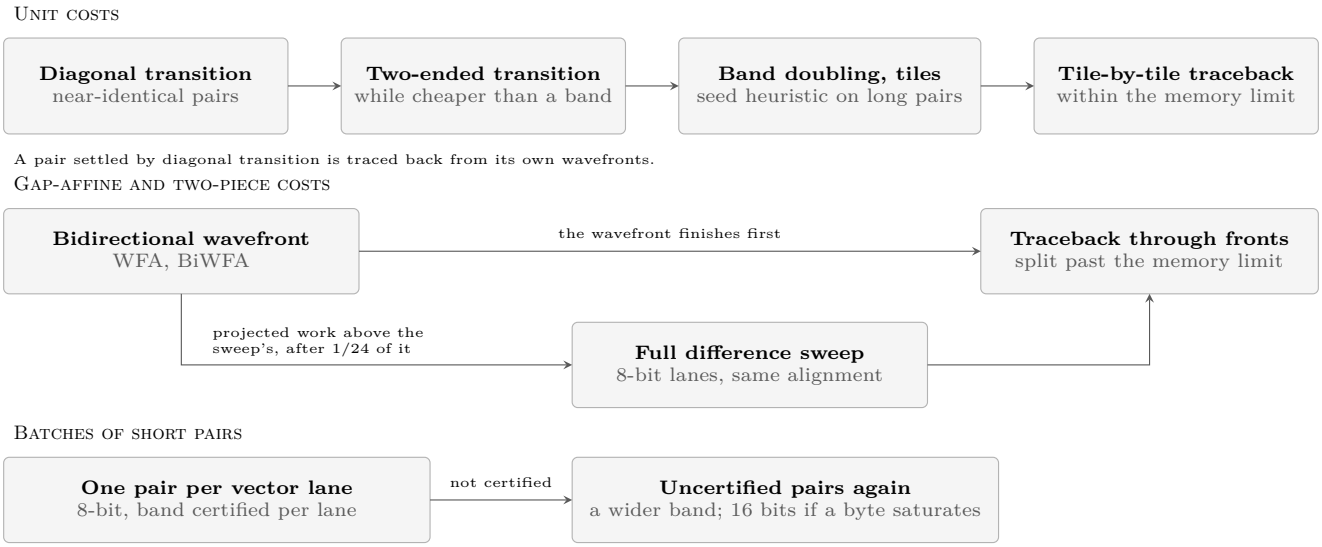


Figure 1: How dinara-align assigns a pair to an algorithm. At unit costs, diagonal transition from one end and then from both settles near-identical pairs while it costs less than a band would; other pairs go to band doubling over bit-parallel tiles, pruned by the seed heuristic on long pairs, and are traced back a tile at a time within the memory limit. At gap-affine costs, the bidirectional wavefront hands a pair to a full sweep in difference recurrences once its projected work exceeds the sweep's, the alignment being the same either way. Batches of short pairs are aligned one pair per vector lane within a band whose optimality each lane certifies.

Table 1: Algorithms in dinara-align and the changes introduced in this work.

Problem	Algorithm	Changes (section)
Unit costs, long pairs	Band doubling over bit-parallel tiles with the gap-chaining seed heuristic (A*PA2)	Regrouped recurrence, staggered groups (3.2); agreeing bounds (3.3); neighbour construction (3.5); traceback within a memory limit (3.7); constant costs per pair (3.11)
Unit costs, similar pairs	Diagonal transition (WFA, BiWFA)	Gathered extension (3.4); giving up on a projection with bounded waste (3.10)
Gap-affine, two-piece	Bidirectional wavefront (WFA, BiWFA); full sweep in difference recurrences for divergent pairs (Suzuki and Kasahara)	Gathered extension (3.4); score-derived band (3.8); one search or two (3.9); handover with bounded waste (3.10)
Substitution matrices, local	Anti-diagonal vector sweep; end-first local alignment (SSW)	Backward wavefront from the best end (3.8)
Batches of short pairs	Inter-sequence vectorisation (SWIPE, SeqAn, parasail)	Certified per-lane bands (3.6)

accesses overlap, and only seeds whose matches could still chain to the end of the pair are considered.

An exact match μ of a seed at query rows $[i, i+16)$ always has four neighbouring matches with one edit: two share its start, at rows $[i, i+15)$ and $[i, i+17)$, and two share its end, at $[i+1, i+16)$ and $[i-1, i+16)$. Because chaining is all-or-nothing, a neighbour one diagonal away may chain where μ cannot, and the neighbours are required for the heuristic to remain a lower bound. Rather than locating them by search and testing each for local pruning, dinara-align inserts them whenever μ is retained, which is justified by the following lemma. Local pruning retains a match ν only if $c(\nu) + h(e(\nu)) < \tau$ (Section 2.4).

Lemma 1. If an exact match μ fails local pruning, so does each of its four neighbours.

Proof. A neighbour ν that shares the end of μ has $e(\nu) = e(\mu)$ and $c(\nu) = 1 > c(\mu) = 0$. A neighbour that shares the start ends on an adjacent diagonal, and one insertion or deletion from $e(\mu)$ reaches a cell on that diagonal at or beyond $e(\nu)$. The least remaining cost does not increase along a diagonal (Ukkonen, 1985), so $h(e(\mu)) \leq 1 + h(e(\nu))$ and $c(\mu) + h(e(\mu)) \leq c(\nu) + h(e(\nu))$. In both cases the pruning test value of ν is at least that of μ. □

Neighbours sharing the start therefore raise the layer of that start by the chain length available from their ends, and neighbours sharing the end are placed one layer below it, without a search or a pruning test of their own.

3.6 Certified bands for inter-sequence batches

Batches of short pairs are aligned with one pair per vector lane (Rognes, 2011; Rahn et al., 2018), first in saturating 8-bit lanes and, for the scores of pairs that saturate, in 16-bit lanes (Zhao et al., 2013; Daily, 2016); alignments stay in 8-bit lanes. Rather than computing each matrix in full, a lane computes the band of diagonals $[d_\ell, d_h]$ containing the main diagonal 0 and the end diagonal $\delta = m - n$, together with one

further diagonal on either side. Its optimality is certified with the following bound, a cost form of the out-of-band bounds of Gibrat (2018) and Fujiki et al. (2020), stated for gap-affine and two-piece costs and with a strict form for tie-breaking.

Proposition 1. Let C be the least cost of an alignment within $[d_\ell, d_h]$, and let $L(d) = g(|d|) + g(|d - \delta|)$. If $C \le \min\big(L(d_h+1), L(d_\ell-1)\big)$, then C is the optimal cost; if the inequality is strict, every optimal alignment lies within the band.

Proof. An alignment that leaves the band visits diagonal $d_h + 1$ or $d_\ell - 1$. Consider $d = d_h + 1 > \max(0, \delta)$; the case below the band is symmetric. Starting on diagonal 0, the alignment contains insertions totalling at least d letters to reach d, and deletions totalling at least $d - \delta$ letters to end on δ. These form at least one gap of each kind, and since g is subadditive (for two-piece costs, with the usual convention that the longer piece has the lower extension cost), their total cost is at least $g(d) + g(d - \delta) = L(d)$. Every alignment outside the band therefore costs at least $\min\big(L(d_h+1), L(d_\ell-1)\big)$. □

Such bounds underlie band doubling (Ukkonen, 1985) and verify narrow-band results in software (Gibrat, 2018) and hardware (Fujiki et al., 2020); here each vector lane of an inter-sequence batch evaluates its own certificate, and the strict form guarantees that the alignment the tie rule selects lies within the band. Pairs that fail, about one in seven in our short-read benchmark, are recomputed over every diagonal a cheaper path could visit, grouped by the width of that band so that the lanes of a group finish together.

3.7 Traceback within a memory limit

The traceback of a band traces each tile from the boundary recorded on its left, the vertical differences down that boundary. On a 1 Mbp pair at 20% errors these boundaries take up to 175 MB, so they are held to the alignment's memory limit, 80 MB by default: beyond it only every k-th is kept, k doubling as the round proceeds. Each tile also records the state the band carries into it, its first and last words, the score at its top, the deepest row kept and its score, and the bound, and the traceback computes a stretch of tiles again from the last kept boundary. Every step of a band depends only on this state and on the scores it computes, so each boundary is recovered as the round left it and the alignment is the same byte for byte; splitting the pair instead, as QuickEd does (Doblas et al., 2025), resolves ties within each part. On the 1 Mbp pairs at 20% errors, the growth of peak memory fell from 175 to 133 MB on the worst pair, for 9% more time; pairs whose boundaries fit are unchanged.

3.8 Gap-affine and score-based alignment

A scoring scheme with match score a, mismatch score b and gap scores $-(o + ke)$ can be converted into costs for global alignment (Eizenga and Paten, 2022). Every alignment covers $n + m$ letters, so its score S and a gap-affine cost C satisfy

$$S = \tfrac{1}{2}\big(a(n+m) - C\big),$$
$$C = 2(a-b)\,X + \sum_{\text{gaps}} \big(2o + k(2e+a)\big), \qquad (4)$$

where X is the number of mismatches, so the wavefront computes score-maximising alignments. The cost also bounds the band in which the alignment must be stored: each gapped letter contributes at least $2e + a$ to C, and an alignment that deviates by t diagonals from both the main and the end diagonal contains at least $2t$ gapped letters, so only diagonals within $C/(2(2e+a))$ of them are stored, 137 instead of 1,000 for a 1 kbp pair at 5% divergence. Local alignment locates the end of the optimal alignment with a forward sweep, as in SSW (Zhao et al., 2013), and extends a wavefront back from it to the first cell attaining the maximal score, which avoids the correction pass that striped layouts may need (Farrar, 2007).

The wavefront keeps five bytes per diagonal for the traceback, the furthest column and one byte naming each layer's source, and a pair whose fronts would exceed the memory limit is split where an optimal path crosses, as in BiWFA. Whether one search or two align a pair is decided while it is aligned (Section 3.9); a cost without traceback stops once its best lies within one of the costs both searches have checked against the start of the other; and a global pair projected to cost more than a full sweep in 8-bit difference recurrences (Suzuki and Kasahara, 2018) is handed to that sweep (Section 3.10), which keeps the predecessor flags of Section 3.1, so that unrelated 5 kbp pairs aligned in 4.8 ms instead of 32.9 ms.

Under a substitution matrix, anti-diagonals are computed in 16-bit lanes while the scores provably fit (Wozniak, 1997), in strips of rows of at most 16 KB that stay in the first-level cache, which took local alignment of 10 kbp against 12 kbp from 50.0 to 17.4 ms. A global alignment is traced in a band of diagonals that grows at most fourfold per step until its score proves that every optimal path lies inside it, so a pair close to its diagonal costs about its own cells: 1 kbp pairs at 10% divergence aligned in about 200 µs instead of 544 µs.

3.9 One search or two at gap-affine costs

WFA grows the wavefront from one end of the pair and keeps every front for the traceback (Marco-Sola et al., 2021); BiWFA grows it from both ends and keeps only the latest fronts (Marco-Sola et al., 2023). For an alignment, two searches must also prove where an optimal path crosses between them, so each must pass half the cost by a margin $m = o + w$, where o is the widest gap opening and w the dearest single move: every optimal path has a cell whose costs from the two ends differ by at most w, and a meeting inside a gap pays its opening once for both halves. With $W(c)$ the number of diagonals a search steps to reach cost c, a pair of cost d costs two searches about $2W((d+m)/2)$ plus the setup of the second, and one search $W(d)$. Where the

fronts widen with the cost, $W(c)$ grows with c^2 and one search is cheaper while $d < (1+\sqrt{2})\,m$; where the ends of the sequences bound the fronts, it is always cheaper. On 250 bp Illumina reads most pairs cost less than m, 14 at costs (4, 6, 2), and two searches took 2.8 µs per read against 1.5 µs for WFA.

dinara-align therefore begins with the one search whose fronts the traceback reads, the backward search for the default tie rule, keeps its fronts, and traces the alignment back as soon as it reaches the far corner. Every cost c passed without reaching it shows that $d > c$, and the search measures W as it grows: it continues alone while $c < m$, costs the second search would pass in any case, and then while $W(c) \le 2W(\lfloor (c+m)/2 \rfloor)$, that is, while it has stepped no more diagonals than two searches would need for the cheapest cost still possible. It also stops at half the memory limit or when a band leaves it no reachable cell. No threshold is tuned: the search hands over at about $2.4m$ where the fronts widen with the cost, and never where the sequences bound them, aligning as WFA does.

Proposition 2. If the first search stops at cost B without reaching the far corner, and the second search, growing from cost 0, checks each new front against the last $w+1$ fronts of the first, costs $B-w$ to B, while its own cost is at most B, then the two searches miss no optimal alignment.

Proof. Let P be an optimal path, of cost d. Having checked at every cost up to B that it had not reached the far corner, where the second search starts, the first search shows that $d > B$. Along P, the cost g from the first search's start never decreases and the cost f from the second's never increases, so $f-g$ never increases. While its cost is at most B, the second search checks each front against costs $B-w$ to B; afterwards the two alternate as in BiWFA, where a cell with $|f-g| \le w$ is checked once both searches have passed it. Since g rises in steps of at most w and $d > B$, some cell of P has $g \in [B-w, B]$. If $f-g < -w$ there, then $f < B-w$, and the cell is checked in the first phase. Otherwise, as $f-g$ never increases, P has a cell with $|f-g| \le w$ and $g \ge B-w$: either $g \le B$ and $f \le B+w$, checked against the costs the first search still holds, or $g > B$, a cell of the alternating phase. In the first case $f < (d+o-w)/2$, as $f+g \le d+o$, and g is already held; in the second both costs are at most $(d+o+w)/2$. The stopping rule ends the searches only once both have passed half the best cost found plus the margin, so the cell is checked first. □

An alignment traced from the unpruned fronts of the single search is the one the two searches would have traced after meeting: the traceback steps only from a cell to a predecessor on an optimal path, and the fronts that the two-search path prunes away hold no such cell, so the tie rule is unchanged. A distance, which needs no traceback, grows the forward search alone through the costs its ring can hold. Starting with one search took a 250 bp Illumina read at (4, 6, 2) from 2.41 to 1.52 µs, and the measured rule saved a further 5%, a third on the pairs costing 9 to 32; nanopore and HiFi reads were unchanged.

3.10 Giving up on a projection, with bounded waste

Two hand-overs rest on projections: diagonal transition gives way to band doubling, and the gap-affine wavefront to the difference sweep, once continuing is projected to cost more than the alternative. Extrapolated from the edits seen so far, a projection is reliable when edits are spread evenly and can exceed the distance a hundredfold when they gather at the ends, as on nanopore reads with adapters or poly-A tails, handing a pair that the search would finish in microseconds to a band over the whole matrix. dinara-align gives up on a projection only once the work already spent has reached one twenty-fourth of the cheapest alternative that could still be needed: for diagonal transition, the band whose bound is the cost already reached, a lower bound on any band because the distance is at least that cost; for the wavefront, the full sweep. Waiting thus costs a search that gives up in the end at most about 4% of what the alternative costs in any case, while a search that would have finished is no longer abandoned on a projection from a handful of edits. For an alignment, diagonal transition also widens a projection from k edits by $1.5/\sqrt{k}$, about its relative error. On nanopore reads at gap-affine costs this took the 10 kbp reads from 2.33 to 1.55 ms and the 50 kbp reads from 30.5 to 21.0 ms (Supplementary Table S1).

3.11 Constant costs on short reads

A 250 bp read aligns in well under a microsecond, so costs that do not depend on the pair dominate. An aligner object keeps every buffer its searches use, the encoded sequences, the rings and the fronts traced back, from pair to pair, one aligner per thread, as each competitor's benchmark runner keeps its own. Both sequences are encoded, and reversed for the leftmost tie rule, in one vectorised pass of 32 bytes with an overlapping tail, and results pass between internal calls as plain values rather than optional ones, which the compiler moves a byte at a time. Together these took a unit-cost alignment of a 250 bp Illumina read through one aligner from 0.97 to about 0.40 µs (Supplementary Table S1).

3.12 Thresholds measured against the code

The choice of algorithm and the bounds of each search rest on 41 thresholds, each set from measurements on the code of its time. We built dinara-align with each halved and with each doubled and timed the 83 builds against the unchanged one (Supplementary Section S1): about thirty moved no sample by more than 3%, and most others traded one dataset against another, but three choices had gone stale, among them the length from which seeds match with one edit, which had cost 100 kbp pairs at 10% errors a third of their running time.

3.13 Validation

A differential fuzzer compares every mode, cost model, band, cost limit and tie rule on random pairs with a full-matrix implementation that shares no code with the library, and the WFA2-lib regression set is reproduced exactly, every CIGAR included (Supplementary Section S1).

4 Results

4.1 Experimental setup

All measurements were taken on an Intel Core i9-7900X (Skylake-X, AVX-512), its ten cores at a fixed 3.3 GHz with turbo boost and hyper-threading disabled, with dinara-align compiled by Mojo 1.1.0 (Modular Inc., 2026) at commit `036e12b` (gap-affine costs at `a6d9166`, the other modes and batches at `289bcf0`) and every tool compiled for the native instruction set. We combined the protocols of A*PA2 (Groot Koerkamp, 2024) and QuickEd (Doblas et al., 2025). The real datasets are A*PA2's nanopore reads of about 1, 10, 50 and 500 kbp, the 500 kbp ones of CHM13 against its own reference and also of a second person against it, which adds genetic variation and gaps of up to 42 kbp; its SARS-CoV-2 genomes; and Illumina 2×250 bp and PacBio HiFi reads, the two technologies QuickEd adds, here reads of HG002 from Genome in a Bottle (Zook et al., 2016) in one region of chromosome 20, each mapped by minimap2 (Li, 2018) to both haplotypes of HG002's own assembly (Hansen et al., 2025) and paired with the stretch it covers, so that a pair differs by sequencing errors alone. The simulated datasets are QuickEd's random sequences of 10 kbp to 1 Mbp whose pairs differ by single-base substitutions, insertions and deletions spread uniformly at 1, 5, 10 and 20%, modelling no sequencing technology. Each dataset was sampled to about 2 Mbp and at least four pairs (Supplementary Table S6), and each tool aligned each pair once, with traceback, on one pinned core, within 30 s per sample, through one aligner object per sample, as every tool's benchmark runner keeps one. The exact aligners compared are A*PA2-full and A*PA2-simple, A*PA (Groot Koerkamp and Ivanov, 2024), Edlib (Šošić and Šikić, 2017), WFA and BiWFA (Marco-Sola et al., 2021, 2023), QuickEd, BSAlign (Shao and Ruan, 2024), KSW2 (Li, 2018), parasail (Daily, 2016) and SSW (Zhao et al., 2013), each on the workloads it supports; every cost agreed with those of every other tool and with the published A*PA2 results.

4.2 Speed and memory

dinara-align had the lowest mean time per alignment of the exact aligners on every dataset. On the real data (Table 2) it was 1.4 to 2.4 times faster than the fastest alternative on the nanopore reads and 2.5 times on the SARS-CoV-2 genomes, and 2.0 and 5.2 times faster than WFA, the fastest of the others there, on the Illumina and HiFi reads. On the simulated data (Fig. 2; Supplementary Table S5) it was 1.1 to 2.6 times faster: 100 kbp pairs at 1, 5, 10 and 20% errors took 1.5, 5.1, 7.3 and 15.7 ms, against 7.5, 8.4, 13.9 and 36.7 ms for A*PA2-full and 12.5, 24.0, 39.0 and 62.8 ms for QuickEd, and its narrowest margin was on 10 kbp pairs at 5%, 0.267 against 0.288 ms for A*PA2-simple. Over all 24 datasets QuickEd took 3.0 to 75 times as long, 8.0 times in the geometric mean. At gap-affine costs dinara-align was 1.1 to 3.8 times faster than WFA and BiWFA wherever a tool completed a dataset (Table 3; Table 4), its narrowest margin on the Illumina reads, 1.45 against 1.55 µs for WFA, and it completed more of the long simulated pairs than any other tool; in the other modes it was 1.0 to 22 times faster than the fastest tool offering the same mode, the one tie lying within the variation between builds. Global scores of 500,000 short-read pairs took 230 ms on one core and 27 ms on ten at (1, 2, 1), 4.6 and 4.3 times faster than WFA2-lib.

An alignment of an ultra-long read increased peak memory by at most 48 MB, against 82 MB for A*PA2-full, and of a 1 Mbp pair at 20% errors by at most 123 MB, against 184 MB, its traceback held within the default limit of 80 MB (Section 3.7; Supplementary Table S7); QuickEd stayed near 25 MB. On short reads dinara-align's peak is higher than the others' (Table 2), its runtime alone taking about 13 MB.

5 Discussion

No single algorithm is fastest across the workloads evaluated here. dinara-align is fastest on all of them because it chooses among algorithms for each pair, and because every choice preserves optimality or bounds its waste, choosing never costs the guarantee or changes the alignment. Three observations stand out. First, on long pairs the techniques that reduce work outside the inner loop contributed more than those that speed it up: constructing seed neighbours directly saved up to 16%, interleaving bit-parallel groups 4.5 to 11.5% and the regrouped recurrence 1 to 4% (Supplementary Table S2). Second, on short reads the costs that do not depend on the pair dominate: before the changes of Section 3.11, a 250 bp alignment at gap-affine costs took a median 4.8 µs through a fresh call and 2.2 µs through a reused aligner, so a benchmark should call every tool as its own runner does. Third, thresholds that encode a cost model go stale as the code changes, and measuring them again belongs in the evaluation of any aligner that selects its algorithm by thresholds (Section 3.12). The measured-work rule, the regrouped recurrence and the per-lane certificate do not depend on this implementation and apply directly to other wavefront, bit-parallel and inter-sequence aligners.

The study has several limitations:

1. The evaluation covers one x86 processor with AVX-512; the AVX2 and NEON configurations are not evaluated here.

2. Some thresholds, in particular the AVX-512 gates for inexact seeds, were set after measurements on uniform 100 kbp pairs like those of Fig. 2a, so

Table 2: Unit-cost alignment of the real datasets by every exact aligner: mean time per alignment in ms, traceback included, and below it the peak resident memory of the run in MB, as QuickEd reports it. t.o. marks a tool that completed no pair within 30 s and o.o.m. one the 8 GB limit stopped. The simulated datasets are in Fig. 2 and Supplementary Table S5.

Dataset	dinara-align	A*PA2-full	A*PA2-simple	A*PA	QuickEd	Edlib	BiWFA	WFA	BSAlign	KSW2
Mean time per alignment										
ont-1k (830 bp)	**0.021**	0.081	0.048	0.612	0.075	0.124	0.044	0.031	0.119	0.539
ont-10k (3.6 kbp)	**0.156**	0.391	0.247	15.3	0.688	1.09	0.741	0.599	1.30	13.1
ont-50k (9.6 kbp)	**0.580**	1.27	0.944	263	2.31	6.14	7.43	6.38	12.3	167
ont-500k (642 kbp)	**118**	283	687	t.o.	847	5,000	6,330	o.o.m.	t.o.	o.o.m.
ont-500k-genvar (663 kbp)	**156**	225	511	t.o.	1,390	5,230	6,200	o.o.m.	t.o.	o.o.m.
SARS-CoV-2 (30 kbp)	**0.286**	2.00	0.728	6.49	3.90	8.18	0.909	0.987	47.1	68.5
Illumina (248 bp)	**0.000576**	0.014	0.00410	0.014	0.016	0.023	0.00204	0.00112	0.021	0.012
PacBio HiFi (14 kbp)	**0.013**	0.977	0.259	1.05	0.950	3.35	0.086	0.066	12.2	3.09
Peak resident memory (MB)										
ont-1k	14	5	5	6	2	7	7	8	2	7
ont-10k	16	6	6	39	6	8	8	39	25	72
ont-50k	19	8	7	858	18	8	10	187	423	650
ont-500k	64	89	90	t.o.	26	15	29	o.o.m.	t.o.	o.o.m.
ont-500k-genvar	64	57	55	t.o.	27	16	27	o.o.m.	t.o.	o.o.m.
SARS-CoV-2	18	7	6	19	15	8	9	26	215	151
Illumina	13	5	5	5	2	7	7	7	2	7
PacBio HiFi	13	7	6	6	4	8	8	9	127	21

Table 3: Gap-affine alignment of the real datasets at costs (4, 6, 2) by every exact aligner that takes them: mean time per alignment in ms, a 30 s budget per sample. Superscripts give the pairs completed where not all were; t.o. marks a tool that completed none and o.o.m. one the 8 GB limit stopped. Every dataset is in Supplementary Table S4.

Dataset	dinara-align	WFA	BiWFA	BSAlign	KSW2
ont-1k	**0.105**	0.188	0.308	0.336	1.10
ont-10k	**1.60**	4.89	5.86	7.84	30.0
ont-50k	**21.5**	63.4	54.9	87.0	$341^{81/104}$
ont-500k	t.o.	o.o.m.	t.o.	o.o.m.	o.o.m.
ont-500k-genvar	$\mathbf{22{,}000}^{1/4}$	o.o.m.	t.o.	o.o.m.	o.o.m.
SARS-CoV-2	**0.710**	2.05	1.78	341	87.9
Illumina	**0.00145**	0.00155	0.00281	0.054	0.013
PacBio HiFi	**0.095**	0.227	0.231	86.0	9.72

results on those pairs may overstate performance on unseen data.

3. As in the A*PA2 protocol, each pair was aligned once; on the Skylake-X some running times varied by up to a third between builds that did not change the code involved, which the ablation controls for (Supplementary Table S2) and the comparison with other tools does not.

4. The Illumina and HiFi reads come from one region of chromosome 20 and exclude reads that minimap2 leaves more than a tenth clipped, such as those spanning structural variants.

5. The ablation switches off one technique at a time, so it does not separate their interactions.

6. Release builds target the baseline processor of each platform, so the AVX-512 code paths require a build for the host processor.

Future work includes:

1. a cheap upper bound on the distance to cap band doubling: a first bound equal to the distance would save up to 27% on long divergent pairs, but a bound 15% too high loses up to 23% on ultra-long reads, so only a bound within a few percent of the distance would pay;

2. reusing computation across band-doubling rounds, as in the incremental doubling of A*PA2, once its exactness for these tiles is proven;

3. faster construction of the seed heuristic, whose scalar lookups dominate on long divergent pairs, and a seed heuristic for gap-affine costs, at which no tool completed a 1 Mbp pair beyond 1% errors;

4. measurements on further processors, and an evaluation of the GPU kernels the library provides for batch scores and alignments (Supplementary Section S1).

Funding

This work received no specific funding.

Conflict of interest

None declared.

Data availability

The software, benchmark harness and the commits of every compared tool are available at `https://github.com/kokyriakidis/dinara-align`. The

Table 4: Gap-affine alignment of QuickEd's simulated datasets, random sequences with uniformly spread substitutions, insertions and deletions, at costs (4, 6, 2) by every exact aligner that takes them: mean time per alignment in ms, a 30 s budget per sample. Superscripts give the pairs completed where not all were; t.o. marks a tool that completed none and o.o.m. one the 8 GB limit stopped. Datasets no tool completed are left out; every dataset is in Supplementary Table S4.

Dataset	dinara-align	WFA	BiWFA	BSAlign	KSW2
10 kbp, 1%	**0.199**	0.416	0.646	37.0	21.3
10 kbp, 5%	**2.32**	7.14	8.72	37.0	96.1
10 kbp, 10%	**7.06**	23.1	27.1	37.3	202
10 kbp, 20%	**20.1**	66.0	75.1	37.4	$375^{81/99}$
100 kbp, 1%	**11.9**	39.9	39.0	o.o.m.	2,420
100 kbp, 5%	**331**	808	714	o.o.m.	$9{,}170^{3/9}$
100 kbp, 10%	**1,160**	o.o.m.	2,430	o.o.m.	$17{,}300^{1/10}$
100 kbp, 20%	**3,610**$^{8/10}$	o.o.m.	$7{,}360^{4/10}$	o.o.m.	o.o.m.
500 kbp, 1%	**477**	1,290	942	o.o.m.	o.o.m.
500 kbp, 5%	**9,910**$^{3/4}$	o.o.m.	$19{,}400^{1/4}$	o.o.m.	o.o.m.
1 Mbp, 1%	**2,300**	o.o.m.	3,940	o.o.m.	o.o.m.

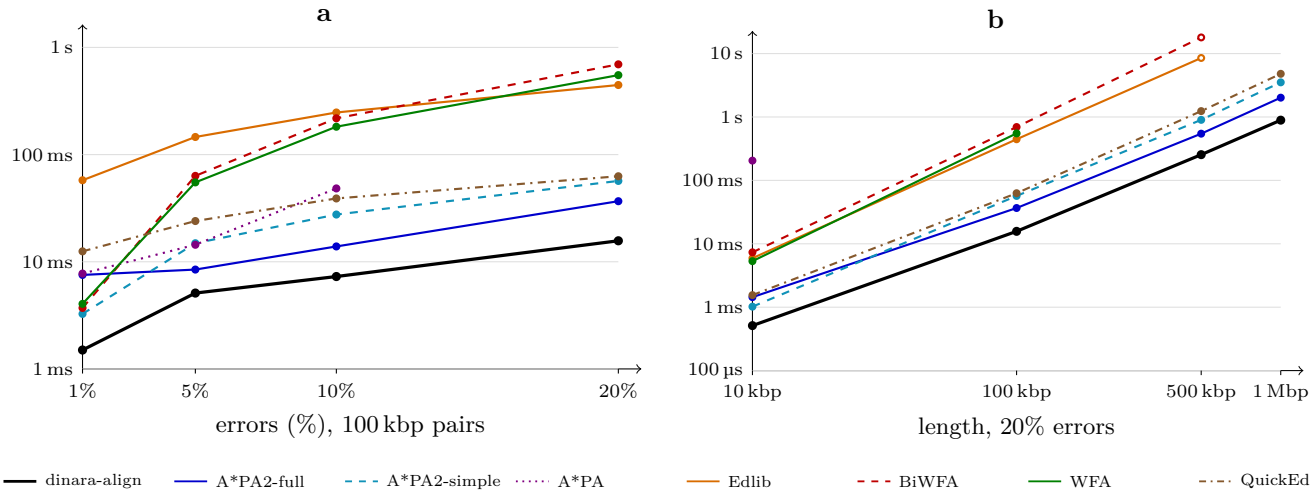


Figure 2: Unit-cost alignment of QuickEd's simulated pairs, mean time per alignment on a logarithmic scale. (a) 100 kbp pairs by error rate. (b) Pairs at 20% errors by length (5% in the interactive version); lines end where a tool completed no pair within 30 s, and hollow points mark tools that completed only some pairs. dinara-align had the lowest mean time at every length and error rate.

benchmarks are reproduced by `pixi run bench` (unit and `-affine` costs), `bench-modes`, `bench-batch` and `bench-ablation`.

References

Jeff Daily. Parasail: SIMD C library for global, semi-global, and local pairwise sequence alignments. *BMC Bioinformatics*, 17:81, 2016. doi: 10.1186/s12859-016-0930-z.

Max Doblas, Oscar Lostes-Cazorla, Quim Aguado-Puig, Nick Cebry, Pau Fontova-Musté, Christopher Batten, Santiago Marco-Sola, and Miquel Moretó. QuickEd: high-performance exact sequence alignment based on bound-and-align. *Bioinformatics*, 41(3):btaf112, 2025. doi: 10.1093/bioinformatics/btaf112.

Jordan M. Eizenga and Benedict Paten. Improving the time and space complexity of the WFA algorithm and generalizing its scoring. bioRxiv, 2022.

Michael Farrar. Striped Smith–Waterman speeds database searches six times over other SIMD implementations. *Bioinformatics*, 23(2):156–161, 2007. doi: 10.1093/bioinformatics/btl582.

Daichi Fujiki, Shunhao Wu, Nathan Ozog, Kush Goliya, David Blaauw, Satish Narayanasamy, and Reetuparna Das. SeedEx: A genome sequencing accelerator for optimal alignments in subminimal space. In *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pages 937–950, 2020. doi: 10.1109/MICRO50266.2020.00080.

Jean-François Gibrat. A short note on dynamic programming in a band. *BMC Bioinformatics*, 19:226, 2018. doi: 10.1186/s12859-018-2228-9.

Osamu Gotoh. An improved algorithm for matching biological sequences. *Journal of Molecular Biology*, 162(3):705–708, 1982. doi: 10.1016/0022-2836(82)90398-9.

Ragnar Groot Koerkamp. A*PA2: Up to 19× faster exact global alignment. In Solon P. Pissis and Wing-Kin Sung, editors, *24th International Workshop on Algorithms in Bioinformatics (WABI 2024)*, volume 312 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 17:1–17:25. Schloss Dagstuhl –

Leibniz-Zentrum für Informatik, 2024. doi: 10.4230/LIPIcs.WABI.2024.17.

Ragnar Groot Koerkamp and Pesho Ivanov. Exact global alignment using A* with chaining seed heuristic and match pruning. *Bioinformatics*, 40(3):btae032, 2024. doi: 10.1093/bioinformatics/btae032.

Nancy F. Hansen, Nathan Dwarshuis, et al. A complete diploid human genome benchmark for personalized genomics. *bioRxiv*, 2025. doi: 10.1101/2025.09.21.677443.

Heng Li. Minimap2: pairwise alignment for nucleotide sequences. *Bioinformatics*, 34(18):3094–3100, 2018. doi: 10.1093/bioinformatics/bty191.

Daniel Liu and Martin Steinegger. Block aligner: an adaptive SIMD-accelerated aligner for sequences and position-specific scoring matrices. *Bioinformatics*, 39(8):btad487, 2023. doi: 10.1093/bioinformatics/btad487.

Santiago Marco-Sola, Juan Carlos Moure, Miquel Moreto, and Antonio Espinosa. Fast gap-affine pairwise alignment using the wavefront algorithm. *Bioinformatics*, 37(4):456–463, 2021. doi: 10.1093/bioinformatics/btaa777.

Santiago Marco-Sola, Jordan M. Eizenga, Andrea Guarracino, Benedict Paten, Erik Garrison, and Miquel Moreto. Optimal gap-affine alignment in $O(s)$ space. *Bioinformatics*, 39(2):btad074, 2023. doi: 10.1093/bioinformatics/btad074.

Modular Inc. Mojo programming language. `https://docs.modular.com/mojo/`, 2026.

Eugene W. Myers. An $O(ND)$ difference algorithm and its variations. *Algorithmica*, 1(1–4):251–266, 1986. doi: 10.1007/BF01840446.

Gene Myers. A fast bit-vector algorithm for approximate string matching based on dynamic programming. *Journal of the ACM*, 46(3):395–415, 1999. doi: 10.1145/316542.316550.

René Rahn, Stefan Budach, Pascal Costanza, Marcel Ehrhardt, Jonny Hancox, and Knut Reinert. Generic accelerated sequence alignment in SeqAn using vectorization and multi-threading. *Bioinformatics*, 34(20):3437–3445, 2018. doi: 10.1093/bioinformatics/bty380.

Torbjørn Rognes. Faster Smith-Waterman database searches with inter-sequence SIMD parallelisation. *BMC Bioinformatics*, 12:221, 2011. doi: 10.1186/1471-2105-12-221.

Haojing Shao and Jue Ruan. BSAlign: a library for nucleotide sequence alignment. *Genomics, Proteomics & Bioinformatics*, 22(2):qzae025, 2024. doi: 10.1093/gpbjnl/qzae025.

Martin Šošić and Mile Šikić. Edlib: a C/C++ library for fast, exact sequence alignment using edit distance. *Bioinformatics*, 33(9):1394–1395, 2017. doi: 10.1093/bioinformatics/btw753.

Hajime Suzuki and Masahiro Kasahara. Introducing difference recurrence relations for faster semi-global alignment of long sequences. *BMC Bioinformatics*, 19(Suppl 1):45, 2018. doi: 10.1186/s12859-018-2014-8.

Esko Ukkonen. Algorithms for approximate string matching. *Information and Control*, 64(1–3):100–118, 1985. doi: 10.1016/S0019-9958(85)80046-2.

Andrzej Wozniak. Using video-oriented instructions to speed up sequence comparison. *Computer Applications in the Biosciences (CABIOS)*, 13(2):145–150, 1997. doi: 10.1093/bioinformatics/13.2.145.

Mengyao Zhao, Wan-Ping Lee, Erik P. Garrison, and Gabor T. Marth. SSW library: An SIMD Smith-Waterman C/C++ library for use in genomic applications. *PLoS ONE*, 8(12):e82138, 2013. doi: 10.1371/journal.pone.0082138.

Justin M. Zook, David Catoe, Jennifer McDaniel, et al. Extensive sequencing of seven human genomes to characterize benchmark reference materials. *Scientific Data*, 3:160025, 2016. doi: 10.1038/sdata.2016.25.